

Universidad Tecnológica Nacional (UTN) - Facultad Regional Mendoza

Carrera: Tecnicatura Universitaria en Programación

Asignatura: Organización Empresarial

Profesor: Francisco Varberde

Trabajo Práctico N° 1: Primeros Pasos con Git

⚠ INSTRUCCIONES ESTRICTAS DE ENTREGA (LEER ATENTAMENTE):

Para que este trabajo práctico sea evaluado, debes entregarlo cumpliendo **exclusivamente** las siguientes condiciones:

1. La entrega final debe ser un único archivo comprimido en formato **.zip**. No se aceptarán formatos como **.rar**, **.7z** ni enlaces web.
2. Dentro del **.zip** debe haber **un documento PDF** con las respuestas a la Parte 1. Este PDF **NO debe contener imágenes ni capturas de pantalla**; solo texto plano y código escrito.
3. Dentro del **.zip** debes incluir la **carpeta oculta** **.git** generada durante la Parte 2. Esta carpeta es esencial para auditar tu historial de comandos, commits y ramas.

El incumplimiento de cualquiera de estas condiciones implica que el trabajo no será tenido en cuenta para la corrección.

Objetivo del Trabajo Práctico

Dar los primeros pasos con Git y comprender por qué es una herramienta esencial para cualquier programador. Al finalizar, sabrás cómo iniciar un proyecto, guardar tus avances de forma segura y usar ramas para probar cosas nuevas sin romper tu código principal.

Parte 1: Evaluación Teórica (Para responder en el PDF)

Responde las siguientes preguntas con tus propias palabras en el documento PDF que incluirás en tu entrega. Apóyate en el material de lectura de las Semanas 1 y 2.

1. ¿Por qué es peligroso gestionar las versiones de tus proyectos simplemente creando carpetas manuales como "proyecto_final", "proyecto_final_v2" o "proyecto_definitivo"? Nombra al menos un riesgo de este método tradicional.
2. En Git, un archivo puede estar en tres estados: Modificado (Modified), Preparado (Staged) o Confirmado (Committed). ¿Qué significa exactamente que un archivo esté "Preparado" (Staged)? Piensa en ello como si estuvieras seleccionando qué objetos meter en una caja antes de cerrarla y enviarla.
3. ¿Para qué sirve el archivo **.gitignore** en un proyecto? Menciona un ejemplo claro de algo que definitivamente NO deberías compartir ni subir a internet.

4. Imagina que estás trabajando en la versión principal de tu programa (tu rama `main`) y se te ocurre probar una idea nueva, pero te da miedo arruinar lo que ya funciona. ¿Cómo te ayuda el concepto de "Ramas" (Branches) de Git a resolver este problema?

Parte 2: Desarrollo Práctico Local

A continuación, realizarás una serie de pasos en la terminal de tu computadora. **No necesitas conexión a internet ni cuenta de GitHub para esta parte.** Todo el trabajo quedará registrado en tu carpeta local `.git`, que actuará como la máquina del tiempo de tu proyecto.

Paso 1: Inicialización y Configuración

- Crea una nueva carpeta en tu computadora llamada `tp1-git-apellido-nombre`.
- Abre tu terminal (Git Bash recomendado en Windows) y navega hasta esa carpeta.
- Inicializa el repositorio de Git.
- Configura tu identidad localmente (solo para este repositorio) con tu nombre y correo electrónico. *Tip: Omite la palabra `--global` en el comando de configuración.*

Paso 2: Primeros Archivos y Guardados (Commits)

- Crea un archivo llamado `README.md` y escribe dentro un título principal y una breve descripción del proyecto usando sintaxis Markdown.
- Crea un archivo llamado `config-secreta.env`.
- Crea el archivo `.gitignore` y configúralo para que Git ignore todos los archivos que terminen en `.env`.
- Utiliza el comando necesario para comprobar el estado de tus archivos (`git status`). Verifica que el archivo secreto ya no aparezca en la lista de archivos sin rastrear (*Untracked*).
- Añade el `README.md` y el `.gitignore` al Área de Preparación (*Staging Area*).
- Realiza tu primer guardado (*commit*) con un mensaje descriptivo y claro (ej: "Añadir estructura inicial y archivo gitignore").

Paso 3: Trabajando en Paralelo (Ramas)

- Crea una nueva rama llamada `desarrollo-funcionalidad` y muévete a ella.
- En esta nueva rama, crea un archivo llamado `app.py` (o usa la extensión del lenguaje que prefieras) y agrega una línea de código simple o un comentario.

- Modifica el archivo `README.md` agregando un nuevo subtítulo que diga "Instrucciones de uso".
- Prepara ambos cambios y realiza un nuevo commit en esta rama.

Paso 4: Uniendo los Caminos (Merge)

- Vuelve a la rama principal (`main` o `master`).
- Observa los archivos en tu carpeta. Notarás que el archivo `app.py` "desapareció" y el `README.md` volvió a su estado original. ¡No te asustes! Git simplemente actualizó tu carpeta para mostrar la versión exacta de tu rama principal.
- Ejecuta el comando para fusionar (integrar) la rama `desarrollo-funcionalidad` hacia tu rama principal. Verás que los archivos nuevos vuelven a aparecer.

Paso 5: Preparación de la Entrega

Al finalizar todos los pasos, tu carpeta local `tp1-git-apellido-nombre` tendrá los archivos de trabajo y la carpeta oculta `.git`. Sigue estos pasos para entregar:

1. Guarda tus respuestas de la Parte 1 en un documento, expórtalo como **PDF** (solo texto, sin imágenes) y colócalo dentro de la carpeta del proyecto.
2. Asegúrate de que la carpeta oculta `.git` siga estando dentro de la carpeta principal.
3. Comprime la carpeta completa en un archivo **.zip**.
4. Sube el archivo `.zip` a la plataforma de la cátedra para su corrección.